Hadoop & Spark - ISEN 2025/2026

Lab: Spark SQL

For this Lab you will need:

- The 3.7 Lab

As a reminder

Check 3.1 to login in NGROK, Kaggle, install Java or Spark and import libraries.

Lab

Make sure we have the taxi dataset correctly loaded: https://www.kaggle.com/datasets/microize/newyork-yellow-taxi-trip-data-2020-2019 (same dataset as a few lab ago, do not download it again if you still have it)

```
taxiSchema = (
                StructType
                ([
                    StructField("VendorId"                , IntegerType()   , True),
                    StructField("lpep_pickup_datetime"    , TimestampType() , True),
                    StructField("lpep_dropoff_datetime"   , TimestampType() , True),
                    StructField("passenger_count"         , DoubleType()    , True),
                    StructField("trip_distance"           , DoubleType()    , True),
                    StructField("RatecodeID"              , DoubleType()    , True),
                    StructField("store_and_fwd_flag"      , StringType()    , True),
                    StructField("PULocationID"            , IntegerType()   , True),
                    StructField("DOLocationID"            , IntegerType()   , True),
                    StructField("payment_type"            , IntegerType()   , True),
                    StructField("fare_amount"             , DoubleType()    , True),
                    StructField("extra"                   , DoubleType()    , True),
                    StructField("mta_tax"                 , DoubleType()    , True),
                    StructField("tip_amount"              , DoubleType()    , True),
                    StructField("tolls_amount"            , DoubleType()    , True),
                    StructField("improvement_surcharge"   , DoubleType()    , True),
                    StructField("total_amount"            , DoubleType()    , True),
                    StructField("congestion_surcharge"    , DoubleType()    , True),
                ])
            )
```

Lab

And read the file from 2020-04:

```
yellowTaxiDF = (
                  spark
                    .read
                    .option("header", "true")
                    .schema(yellowTaxiSchema)
                    .csv("yellow_tripdata_2020-04.csv")
               )

yellowTaxiDF.printSchema()
```

Lab

Let's create a SQL view of this dataset:

```
yellowTaxiDF.createOrReplaceTempView("YellowTaxis")
```

Lab

Let's start simple with a select:

```
outputDF = spark.sql(
                        "SELECT * FROM YellowTaxis WHERE PULocationID = 171"
                    )

outputDF.show()
```

Lab

We will try a merge on a similar dataset, green taxis:

https://www.kaggle.com/datasets/jinbonnie/2020-green-taxi-trip-data-in-new-york-city

```
!kaggle datasets download -d jinbonnie/2020-green-taxi-trip-data-in-new-york-city
!unzip 2020-green-taxi-trip-data-in-new-york-city.zip
```

```
greenTaxiDF = (
                  spark
                    .read
                    .option("header", "true")
                    .csv("2020_Green_Taxi_Trip_Data__January_-_June_.csv")
              )

greenTaxiDF.createOrReplaceTempView("GreenTaxis")
```

Lab

Let's do our merge between both datasets in SQL and create a view:

```
merge_Green_and_Yellow = spark.sql("""

SELECT 'Yellow'                   AS TaxiType

      , lpep_pickup_datetime      AS PickupTime
      , lpep_dropoff_datetime     AS DropTime
      , PULocationID              AS PickupLocationId
      , DOLocationID              AS DropLocationId
FROM YellowTaxis

UNION ALL

SELECT 'Green'                    AS TaxiType

      , lpep_pickup_datetime      AS PickupTime
      , lpep_dropoff_datetime     AS DropTime
      , PULocationID              AS PickupLocationId
      , DOLocationID              AS DropLocationId
FROM GreenTaxis

""")
merge_Green_and_Yellow.createOrReplaceTempView("merge_Green_and_Yellow")
merge_Green_and_Yellow.show()
```

Lab

Show the green ones to be sure they are there:

```
spark.sql("SELECT * FROM merge_Green_and_Yellow WHERE TaxiType = 'Green'").show()
```

Lab

Let's read the taxi zones available in the Yellow taxi Kaggle Dataset:

```
taxiZonesSchema = "LocationID INT, Borough STRING, Zone STRING, ServiceZone STRING"

taxiZonesDF = (
                  spark
                    .read
                    .schema(taxiZonesSchema)
                    .option("header", "true")
                    .csv("taxi+_zone_lookup.csv")
               )
```

And create global temp view:

```
taxiZonesDF.createOrReplaceGlobalTempView("TaxiZones")

taxiZonesDF.show()
```

Exercise 1

Let's create a small report:
Figure out number of rides, grouped by Borough and type of Taxi.

Example:

Borough	TaxiType	TotalTrips
Borough 1	Green	102
Borough 1	Yellow	227
Borough 2	Green	58
Borough 2	Yellow	99

This is a hard exercise, do not hesitate to look further in the documentation for SQL syntax and take a look at the hints on next slide.

Exercise 1

Hints:

- Create a select on the interesting columns from the TaxiZones view we created:
  - Use 'FROM global_temp.TaxiZones' to designate the temp view.
  - Use the COUNT(*) function to count the total number of rides for each group (Borough and TaxiType).
- Use the UNION ALL clause to join the YellowTaxis and GreenTaxis datasets, and add a new column "TaxiType" to distinguish between the two types of taxis (if not already done previously).
  - For this you can use TaxiType, PULocationID columns from YellowTaxis & GreenTaxis
  - You wil later do a match between PULocationID from the taxis and LocationID from the TaxiZones
- Perform a LEFT JOIN on the TaxiZones dataset with the combined taxi dataset (Yellow and Green taxis) using the appropriate columns (PULocationID and LocationID).
- Use the GROUP BY clause to group the results by Borough and TaxiType.
- Finally, use the ORDER BY clause to sort the results by Borough and TaxiType.